

Strings - Aula 1

Autor: Samuel Raimundo
samuel.r.pinto@ufv.br

Sumário

- Função Z
- Hash
- Utilitários

Função Z

O que é a Z-Function?

Para uma string S de tamanho n , seu **Z-Array** é um array de inteiros de tamanho n .

$Z[i]$ é o comprimento do **maior prefixo comum** entre a string S inteira e o sufixo de S que começa na posição i .

Formalmente: $Z[i] = \text{LCP}(S, S.\text{substr}(i))$

Convenção: $Z[0]$ é geralmente definido como 0, pois não é utilizado nos cálculos.

Calculando o Z-Array: Um Exemplo

String **S** = **ABACABA** (tamanho $n=7$)

Calculando o Z-Array: Um Exemplo

String **S** = **ABACABA** (tamanho $n=7$)

$Z[0] = 0$ (por convenção)

$i=1$: S vs BACABA $\rightarrow$ LCP = 0. $Z[1] = 0$

$i=2$: S vs ACABA $\rightarrow$ LCP = "A" (comp. 1). $Z[2] = 1$

$i=3$: S vs CABA $\rightarrow$ LCP = 0. $Z[3] = 0$

$i=4$: S vs ABA $\rightarrow$ LCP = "ABA" (comp. 3). $Z[4] = 3$

$i=5$: S vs BA $\rightarrow$ LCP = 0. $Z[5] = 0$

$i=6$: S vs A $\rightarrow$ LCP = "A" (comp. 1). $Z[6] = 1$

Resultado:

S: |A|B|A|C|A|B|A|

Z: |0|0|1|0|3|0|1|

Aplicação: Busca de Padrões (Pattern Matching)

Problema: Encontrar todas as ocorrências de um padrão P em um texto T .

Aplicação: Busca de Padrões (Pattern Matching)

Problema: Encontrar todas as ocorrências de um padrão P em um texto T .

A Estratégia com Z-Function:

1. Crie uma nova string: $S = P + '$' + T$
 - ($\$$ é um caractere separador que não existe em P ou T).
2. Calcule o Z-Array para S .
3. Uma ocorrência do padrão P é encontrada sempre que um $Z[i]$ for igual ao comprimento de P .
4. A posição da ocorrência no texto T será: $i - |P| - 1$.

Aplicação: Busca com um Erro

Problema: Encontrar **P** em **T** com no máximo **um** caractere de diferença.

Aplicação: Busca com um Erro

Problema: Encontrar P em T com no máximo **um** caractere de diferença.

A Ideia: Um match com erro consiste em um **prefixo** correto de P seguido por um **sufixo** correto de P .

A Estratégia:

1. **Prefixos:** Calcule o Z-Array Z_fwd para $S1 = P + '$' + T$.
2. **Sufixos:** Calcule o Z-Array Z_bwd para $S2 = reverse(P) + '$' + reverse(T)$.
3. **Verificação:** Para cada posição k no texto T , verificamos se $comprimento_prefixo + comprimento_sufixo \geq |P| - 1$.
 - O comprimento do prefixo vem de Z_fwd .
 - O comprimento do sufixo vem de Z_bwd .

Outras Aplicações

Contagem de Substrings Distintas

Outras Aplicações

Contagem de Substrings Distintas

- **Ideia:** Processar a string de forma online (caractere a caractere).
- **Método:** Ao adicionar um caractere, o número de *novas* substrings é $\text{tamanho_atual} - \text{LCP}$, onde LCP é o comprimento da maior substring terminada no novo caractere que já apareceu antes.
- Este LCP pode ser encontrado com $\max(\text{Z}(\text{reverse}(\text{S_nova})))$.

Outras Aplicações

Contagem de Substrings Distintas

- **Ideia:** Processar a string de forma online (caractere a caractere).
- **Método:** Ao adicionar um caractere, o número de *novas* substrings é $\text{tamanho_atual} - \text{LCP}$, onde LCP é o comprimento da maior substring terminada no novo caractere que já apareceu antes.
- Este LCP pode ser encontrado com $\max(\text{Z}(\text{reverse}(\text{S_nova})))$.

Periodicidade de uma String

Outras Aplicações

Contagem de Substrings Distintas

- **Ideia:** Processar a string de forma online (caractere a caractere).
- **Método:** Ao adicionar um caractere, o número de *novas* substrings é $\text{tamanho_atual} - \text{LCP}$, onde LCP é o comprimento da maior substring terminada no novo caractere que já apareceu antes.
- Este LCP pode ser encontrado com $\max(\text{Z}(\text{reverse}(\text{S_nova})))$.

Periodicidade de uma String

- **Problema:** Encontrar a menor string T tal que $S = T + T + \dots + T$.
- **Método:** O menor k que divide o tamanho de S e satisfaz a condição $k + \text{Z}[k] == n$ nos dá o comprimento do período.
- **Exemplo:** $S = \text{ababab}$. $n=6$, $k=2$. $\text{Z}[2]=4$. $2 + \text{Z}[2] = 6$. Período é "ab".

Implementação trivial

- Dado um sufixo $S[i : n-1]$, vamos calcular o maior prefixo de $S[i : n-1]$ que também é prefixo de S .

```
1  vector<int> z_function_trivial(string s) {  
2      int n = s.size();  
3      vector<int> z(n);  
4      for (int i = 1; i < n; i++) {  
5          while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {  
6              z[i]++;  
7          }  
8      }  
9      return z;  
10 }
```

Implementação Eficiente

- Implementação em $O(N)$

```
1  vector<int> get_z(string s) {  
2      int n = s.size();  
3      vector<int> z(n, 0);  
4      int l = 0, r = 0;  
5      for (int i = 1; i < n; i++) {  
6          if (i <= r) z[i] = min(r - i + 1, z[i - 1]);  
7          while (i + z[i] < n and s[z[i]] == s[i + z[i]]) z[i]++;  
8          if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;  
9      }  
10     return z;  
11 }
```


Hash

O que é hash?

- Transformar um elemento complexo (string) em um chave menor.
- Vamos denotar a função de hash de uma string **T** por **H(T)**.
- Idealmente desejamos que **H(S) = H(T) $\Leftrightarrow$ S = T**.
- Na prática temos **S = T $\Rightarrow$ H(S) = H(T)**.
- Portanto, desejamos que se **H(S) = H(T)**, então **P(S = T) $\approx$ 1**.
- Em outras palavras, devemos evitar colisão de valores de hash para strings distintas.
- Opção: Usar múltiplas hashes.

Rabin-Karp

- Ocorrências de **S** em **T**.
- Calcula a Rolling Hash de **T**.
- Calcula a hash de **S**.
- Complexidade **$O(|S| + |T|)$**

$$H(S) = H(T_{i,i+|S|-1}) \quad \forall i \in \{0, 1, 2, \dots, |T| - |S|\}$$

Análise

- A complexidade de criar é **$O(N)$** .
- A complexidade de acesso a um intervalo é **$O(1)$** .
- Por que não usar sempre?
 - Colisões.
 - Constante alta (muitas operações modulares).
 - Nem sempre resolve o problema.

Utilitários

Manacher

- Se $S[i:j]$ é palíndromo.
- Maior palíndromo que termina em cada posição.

Min/Max Cyclic Shift

- Computa o índice do menor/maior ciclo lexicográfico.

S = babaabcd

min = aabcdbab

max = dbabaabc



Dúvidas?